

Build a production-style Smart Parking Management System as a Final Year Computer Science Project.

Tech Stack:

Frontend:

- React.js with Vite
- React Router
- Context API
- Socket.IO Client

Backend:

- Node.js
- Express.js
- Socket.IO

Authentication:

- Firebase Authentication
- Email and Password login
- Protected routes

Database:

- PostgreSQL

Object Detection:

- Python
- OpenCV
- YOLOv8
- Camera-based parking occupancy detection

Project Requirements:

Parking Structure:

- Three parking zones:
 - Zone A
 - Zone B
 - Zone C
- Each zone contains 15 parking slots.
- Total slots = 45.

Booking Logic:

- User books a zone instead of an individual slot.
- System automatically assigns an available slot inside the selected zone.
- If no slot is available, booking should be rejected.

User Features:

- Register and Login using Firebase Authentication.
- View available slots in each zone.
- View real-time occupancy.
- Create booking.
- Cancel booking.
- View booking history.
- View active booking.
- View parking duration.
- View parking charges.

Parking Rules:

- Charge ₹20 per hour.
- Parking duration is calculated using entry and exit timestamps.
- Billing should be generated automatically.

Admin Features:

- View occupancy of all zones.
- View total bookings.
- View active bookings.
- View completed bookings.
- View revenue analytics.
- View parking utilization analytics.
- View object detection logs.
- Force-release occupied slots if required.

Object Detection Requirements:

- Use camera feeds.
- Use YOLOv8 to detect vehicles.
- Detect:
 - Car
 - Bike
 - Truck
- Map detections to parking slots.
- Update slot status automatically.
- Send updates to backend.
- Store occupancy history.

Realtime Requirements:

- Use Socket.IO.
- Occupancy updates should appear instantly on all connected clients.
- Booking status updates should appear instantly.
- Dashboard analytics should refresh automatically.

Database Design: Create normalized PostgreSQL schema including:

- users
- zones
- parking_slots
- bookings
- occupancy_logs
- payments
- detection_logs

API Design: Generate REST APIs for:

- authentication
- bookings
- slots
- zones
- payments
- admin dashboard
- analytics

Frontend Pages:

- Login Page
- Register Page
- Home Dashboard
- Zone Selection Page
- Booking Page
- Active Booking Page
- Booking History Page
- User Profile Page
- Admin Dashboard
- Analytics Dashboard

Architecture: Provide:

- Folder structure
- Database schema
- API documentation
- React component hierarchy
- Socket.IO flow
- YOLO integration flow

- Deployment plan

Generate the project phase by phase rather than producing the entire implementation at once.